Practical No. 3

Mapped CO: CO2 Units III & IV Bloom Level: L6 – Create

1.0) AIM

Write, compile, and deploy a simple smart contract in the Solidity programming language using the Remix IDE, and interact with it to store and retrieve a value.

2.0) SCOPE

To understand how a smart contract is written in Solidity, compiled, and deployed, and how its functions are used to write data to and read data from the blockchain.

3.0) FACILITIES / APPARATUS

- **Computer System:** Any desktop or laptop computer with an internet connection and a web browser.
- **Software:** Remix IDE (used online at **https://remix.ethereum.org —** no installation required). Optional: MetaMask + Ganache for the extension activity.

4.0) THEORY

What is a Smart Contract?

A smart contract is a program that is stored and runs on a blockchain. Once deployed, its code cannot be changed, and it runs exactly as written — no single person can alter its behaviour. Smart contracts are used to hold data and rules that everyone on the network can trust. On Ethereum, smart contracts are most commonly written in the Solidity language.

What is Solidity?

Solidity is a high-level programming language designed for writing Ethereum smart contracts. Its syntax is similar to JavaScript and C++. A Solidity program can contain state variables (data stored permanently on the blockchain) and functions (which read or change that data).

What is Remix IDE?

Remix is a free, browser-based development environment for Solidity. It lets you write, compile, deploy, and test smart contracts without installing anything. Remix also provides a built-in test blockchain called the Remix VM, so contracts can be deployed and tested instantly and for free.

Understanding the Simple Storage contract:

This practical uses a small contract called SimpleStorage. It stores a single number and provides two functions: one to set (write) a new value and one to get (read) the stored value. This demonstrates the two most fundamental operations of any smart contract — writing data to the blockchain and reading it back.

- A state variable storedNumber holds the value permanently on the blockchain.
- The set() function changes the stored value (this is a transaction that writes to the blockchain).
- The get() function returns the stored value (this only reads data, so it is free and instant).

5.0) PROGRAM (Solidity — SimpleStorage.sol)

```
// SPDX-License-Identifier: MIT
```

```
pragma solidity ^0.8.0;

contract SimpleStorage
{
    // State variable: stored permanently on the blockchain
    uint256 storedNumber;

    // set() writes a new value (a transaction)
    function set(uint256 newNumber) public
     {
        storedNumber = newNumber;
     }

    // get() reads the stored value (free, no transaction)
    function get() public view returns (uint256)
     {
        return storedNumber;
     }
}
```

6.0) PROCEDURE / STEPS TO RUN

1. **Open Remix:** Go to https://remix.ethereum.org in your browser.
2. **Create the file:** In the File Explorer (left panel), open the 'contracts' folder and create a new file named SimpleStorage.sol. Type or paste the program above.
3. **Compile:** Click the 'Solidity Compiler' tab (left side). Select a compiler version 0.8.x and click 'Compile SimpleStorage.sol'. A green tick means it compiled successfully.
4. **Deploy:** Click the 'Deploy & Run Transactions' tab. In the ENVIRONMENT dropdown, select 'Remix VM (Cancun)'. Click the orange 'Deploy' button.
5. **Find the deployed contract:** Under 'Deployed Contracts' (bottom-left), expand the SimpleStorage contract to see its set and get buttons.
6. **Store a value:** Next to 'set', type a number (e.g., 25) and click 'set'. This runs a transaction that writes the value.
7. **Retrieve the value:** Click the blue 'get' button. It returns the stored value (25), read directly from the blockchain.

7.0) EXPECTED OUTPUT

After deployment, the SimpleStorage contract appears under 'Deployed Contracts' with set and get buttons. Calling set(25) runs a transaction (shown as a green tick in the Remix console). Calling get() then returns the stored value:

```
Deployed Contracts
  SimpleStorage at 0xd91...39138 (memory)

  set  25        ->  transaction successful (green tick in console)
  get            ->  0: uint256: 25
```

Observation: *set() creates a transaction because it changes data on the blockchain, while get() is free and instant because it only reads data. The stored value stays on the blockchain until changed by another set().*

Optional Extension — Deploy via MetaMask + Ganache

The steps above deploy to Remix's built-in test blockchain. To instead deploy through a real wallet to

your local Ganache blockchain (from Practical 0), do the following:

1. Start Ganache (Quickstart) and make sure MetaMask is connected to the Ganache Local network with an imported account (see Practical 0).
2. In Remix, on the 'Deploy & Run Transactions' tab, change the ENVIRONMENT dropdown to 'Injected Provider – MetaMask'.
3. MetaMask will pop up asking to connect — approve it. Remix now shows your Ganache account and its ETH balance.
4. Click 'Deploy'. A MetaMask window appears asking you to confirm the transaction (and its gas fee). Click 'Confirm'.
5. The contract deploys to Ganache. Open the Ganache window → 'Transactions' tab to see the deployment recorded there. Now set() will also require a MetaMask confirmation, because it is a real transaction on the Ganache blockchain.

This shows the difference between Remix's simulated blockchain and a real wallet-signed deployment — exactly what happens on the public Ethereum network, just on your local machine.

8.0) CONCLUSION

In this way, we successfully wrote, compiled, and deployed a smart contract (SimpleStorage) using the Solidity language in the Remix IDE, and used its set() and get() functions to write data to and read data from the blockchain — demonstrating the fundamental working of an Ethereum smart contract.